
Week 2 — D3 Scales Code Reference

Setup

For these examples, D3 is imported at the top of each script block:

```
JavaScript
<script>
  import * as d3 from 'd3'
</script>
```

Make sure D3 is installed in your project first:

```
Shell
npm install d3
```

What a scale is

A scale is a function. Give it a value from your data's world, it returns a value from your visual world.

```
JavaScript
import * as d3 from 'd3'

const scale = d3.scaleLinear()
  .domain([0, 100])    // data goes from 0 to 100
  .range([0, 500])     // pixels go from 0 to 500

scale(0)    // 0
scale(50)   // 250
scale(100)  // 500
```

scaleLinear

For quantitative data. Temperature, income, population — anything you'd measure.

None

```
<script>
  import * as d3 from 'd3'

  const data = [
    { city: "New York",    temp: 84 },
    { city: "Philadelphia", temp: 91 },
    { city: "Chicago",     temp: 78 },
    { city: "Los Angeles", temp: 72 },
    { city: "Houston",     temp: 96 }
  ]

  const width = 600
  const height = 400
  const margin = { top: 20, right: 20, bottom: 40, left: 50 }

  const yScale = d3.scaleLinear()
    .domain([0, 100])
    .range([height - margin.bottom, margin.top])
</script>

<svg viewBox="0 0 {width} {height}" {width} {height}>
  {#each data as d, i}
    <rect
      x={margin.left + i * 100}
      y={yScale(d.temp)}
      width="60"
      height={height - margin.bottom - yScale(d.temp)}
      fill="steelblue"
    />
  {/each}
</svg>
```

Notice the range is `[height - margin.bottom, margin.top]` — inverted. Higher data values should appear higher on the canvas, but SVG counts from the top. Inverting the range handles that.

`yScale(d.temp)` gives you the y position of the top of the bar. `height - margin.bottom - yScale(d.temp)` gives you the height — the distance from that position down to the bottom of the chart area.

Deriving domain from your data

Hard-coding the domain to `[0, 100]` only works if you know your data will always fall in that range. For real datasets, derive it:

None

```
<script>
  import * as d3 from 'd3'

  const data = [
    { city: "New York",    temp: 84 },
    { city: "Philadelphia", temp: 91 },
    { city: "Chicago",     temp: 78 },
    { city: "Los Angeles", temp: 72 },
    { city: "Houston",     temp: 96 }
  ]

  const width = 600
  const height = 400
  const margin = { top: 20, right: 20, bottom: 40, left: 50 }

  const yScale = d3.scaleLinear()
    .domain([0, d3.max(data, d => d.temp)])
    .range([height - margin.bottom, margin.top])
</script>
```

`d3.max(data, d => d.temp)` finds the highest temp value automatically. A few helpers worth knowing:

JavaScript

```
d3.min(data, d => d.temp)    // 72
d3.max(data, d => d.temp)    // 96
d3.extent(data, d => d.temp) // [72, 96]
```

`d3.extent` returns both min and max in one call — useful when you need both, which is common for line charts and scatter plots.

The domain starts at 0 rather than `d3.min(...)`. For column and bar charts, bars should extend from zero — using the data minimum as the baseline makes small differences look much larger than they are.

scaleBand

For categorical data on an axis. Divides the available space into evenly-sized bands, one per category.

None

```
<script>
  import * as d3 from 'd3'

  const data = [
    { city: "New York",    temp: 84 },
    { city: "Philadelphia", temp: 91 },
    { city: "Chicago",     temp: 78 },
    { city: "Los Angeles", temp: 72 },
    { city: "Houston",     temp: 96 }
  ]

  const width = 600
  const height = 400
  const margin = { top: 20, right: 20, bottom: 40, left: 50 }

  const xScale = d3.scaleBand()
    .domain(data.map(d => d.city))
    .range([margin.left, width - margin.right])
```

```

        .padding(0.2)

const yScale = d3.scaleLinear()
    .domain([0, d3.max(data, d => d.temp)])
    .range([height - margin.bottom, margin.top])
</script>

<svg viewBox="0 0 {width} {height}" {width} {height}>
  {#each data as d}
    <rect
      x={xScale(d.city)}
      y={yScale(d.temp)}
      width={xScale.bandwidth()}
      height={height - margin.bottom - yScale(d.temp)}
      fill="steelblue"
    />
  {/each}
</svg>

```

`scaleBand` gives you two things:

- `xScale(d.city)` — the x position of the left edge of that column
- `xScale.bandwidth()` — the width of each column, calculated automatically

`padding(0.2)` adds gaps between columns. The value is a proportion of the band width — 0 is no gaps, 1 is no columns. Between 0.1 and 0.3 usually looks right.

scaleOrdinal

For mapping categories to colors or other discrete values.

```

None

<script>
  import * as d3 from 'd3'

```

```

const data = [
  { city: "New York",      temp: 84, region: "Northeast" },
  { city: "Philadelphia", temp: 91, region: "Northeast" },
  { city: "Chicago",      temp: 78, region: "Midwest"   },
  { city: "Los Angeles",  temp: 72, region: "West"      },
  { city: "Houston",      temp: 96, region: "South"     }
]

const width = 600
const height = 400
const margin = { top: 20, right: 20, bottom: 40, left: 50 }

const xScale = d3.scaleBand()
  .domain(data.map(d => d.city))
  .range([margin.left, width - margin.right])
  .padding(0.2)

const yScale = d3.scaleLinear()
  .domain([0, d3.max(data, d => d.temp)])
  .range([height - margin.bottom, margin.top])

const colorScale = d3.scaleOrdinal()
  .domain(["Northeast", "Midwest", "West", "South"])
  .range(d3.schemeTableau10)
</script>

<svg viewBox="0 0 {width} {height}" {width} {height}>
  {#each data as d}
    <rect
      x={xScale(d.city)}
      y={yScale(d.temp)}
      width={xScale.bandwidth()}
      height={height - margin.bottom - yScale(d.temp)}
      fill={colorScale(d.region)}
    />
  {/each}

```

```
</svg>
```

`d3.schemeTableau10` is a built-in array of ten distinct colors. Other options worth knowing:

JavaScript

```
d3.schemeTableau10 // 10 colors, good general purpose
d3.schemeSet2       // 8 softer colors
d3.schemeCategory10 // the classic D3 palette
```

Or define your own:

None

```
<script>
  import * as d3 from 'd3'

  const colorScale = d3.scaleOrdinal()
    .domain(["Northeast", "Midwest", "West", "South"])
    .range(["#4e79a7", "#f28e2b", "#e15759", "#76b7b2"])
</script>
```

scaleTime

For temporal data. Works like `scaleLinear` but understands JavaScript `Date` objects.

None

```
<script>
  import * as d3 from 'd3'

  const data = [
    { date: "2020-01-01", value: 120 },
    { date: "2021-01-01", value: 145 },
    { date: "2022-01-01", value: 98  },
    { date: "2023-01-01", value: 162 },
    { date: "2024-01-01", value: 180 }
  ]
```

```

]

const width = 600
const height = 400
const margin = { top: 20, right: 20, bottom: 40, left: 50 }

// dates in a CSV come in as strings – convert them
const parsed = data.map(d => ({ ...d, date: new Date(d.date) }))

const xScale = d3.scaleTime()
  .domain(d3.extent(parsed, d => d.date))
  .range([margin.left, width - margin.right])

const yScale = d3.scaleLinear()
  .domain([0, d3.max(parsed, d => d.value)])
  .range([height - margin.bottom, margin.top])

const lineGenerator = d3.line()
  .x(d => xScale(d.date))
  .y(d => yScale(d.value))
</script>

<svg viewBox="0 0 {width} {height}" {width} {height}>
  <path
    d={lineGenerator(parsed)}
    fill="none"
    stroke="steelblue"
    stroke-width="2"
  />

  {#each parsed as d}
    <circle
      cx={xScale(d.date)}
      cy={yScale(d.value)}
      r="5"
      fill="steelblue"

```

```
    />
  {/each}
</svg>
```

`d3.extent` is a natural fit for `scaleTime` — you almost always want the full range of your dates as the domain.

The parsing step is important. CSV data comes in as strings — `"2020-01-01"` — and `scaleTime` needs actual `Date` objects. `new Date(d.date)` handles the conversion. If your dates aren't rendering correctly, that's almost always why.

The full pattern

None

```
<script>
  import * as d3 from 'd3'

  // 1. data
  const data = [...]

  // 2. dimensions
  const width = 600
  const height = 400
  const margin = { top: 20, right: 20, bottom: 40, left: 50 }

  // 3. scales
  const xScale = d3.scaleBand() // or d3.scaleTime() or
d3.scalePoint()
    .domain(...)
    .range([margin.left, width - margin.right])

  const yScale = d3.scaleLinear()
    .domain([0, d3.max(data, d => d.value)])
    .range([height - margin.bottom, margin.top])
```

```
// 4. generators (if needed)
const lineGenerator = d3.line().x(...).y(...)
</script>

<!-- 5. rendering -->
<svg viewBox="0 0 {width} {height}" {width} {height}>
  <!-- marks -->
</svg>
```

D3 handles steps 3 and 4. Svelte handles step 5.